3/04/26

OOP II - Access Modifiers and Constructors

Rewriting Notes from Memory

→ In Java, by the default paradigm, uses only pass by value. It does not have native support for pass by reference as observed in languages like C++ etc., where you can directly send pointers

```
function fun(int n) {
    // ...
}
```

→ This function creates a copy of the parameter variable by creating a variable on the stack

```
function fun(int &n) {
    // ...
}
```

→ This function directly mutates the passed variable, and does not create a new variable on the stack. This is NOT SUPPORTED in Java.

```
function main() {
    fun(5);
}
```

Shallow Copy

→ surface level copy of an object

→ default behaviour

```
class Student {
    private String name;
    private int age;

    public Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
```

→ Using the above class, let's create objects

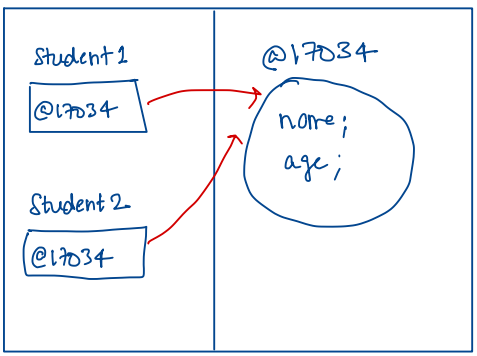
```
Student student1 = new Student("John Doe", 24);
Student student2 = student1;
```

→ The highlighted line results in a shallow copy, due to the default nature of Java, i.e pass by value.

→ So, if student1 holds @17034 (address), now student2 also holds the value @17034

Disadvantage

→ Making any change in student 2 will mutate the state of student 1 as well (as both point to the same address in memory)



Deep Copy

→ The above scenario should be avoided using a copy constructor

```
public Student (Student otherStudent) {
    this.name = otherStudent.name;
    this.age = otherStudent.age;
}
```

→ This is how we should create student 2

```
Student student2 = new Student (student1);
```

→ Here student 2 is a completely different object, while having the same values as that of student 1

```
class Exam {
    int id;
    int marks;

    public Exam(int id, int marks) {
        this.id = id;
        this.marks = marks;
    }
}

class Student {
    private String name;
    private int age;
    private Exam exam;

    public Student(String name, int age, Exam exam) {
        this.name = name;
        this.age = age;
        this.exam = exam;
    }
```

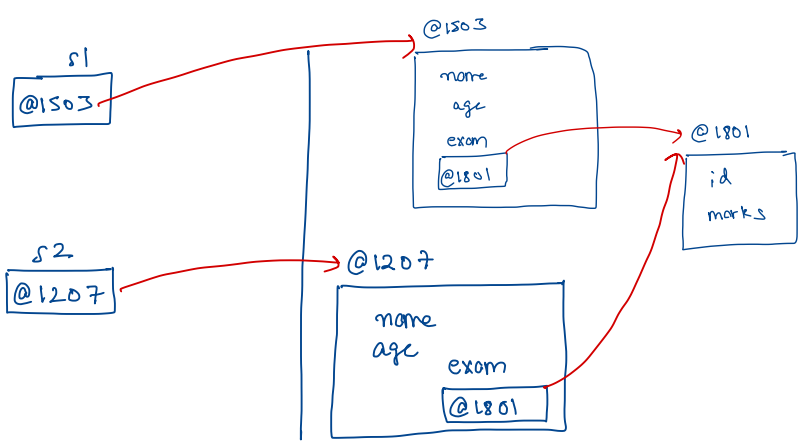
```
    public Student ( Student student ) {
        this. name = student. name;
        this. age = student. age;
        this. exam = student. exam
    }
}
```

For Exam & Student classes

```
Exam exam = new Exam ( 189302001, 88);
Student s1 = new Student ("John Doe", 23, exam);
Student s2 = new Student (s1);
```

→ Since s2 is created using the copy constructor, a deep copy would have been created right?

→ No, because of this. exam = student. exam assignment, which resulted in a shallow copy



→ So, even though s1 & s2 are different objects in memory, they are interlinked as they have a shared state.

→ Changing the exam property in s1 would change the exam property for s2 as well, which is not the desired behaviour.

Fix

1) Exam Copy Constructor

```
public Exam ( Exam exam) {
    this.id = exam.id;
    this.marks = exam.marks;
}
```

2) Student Copy Constructor (Recommended)

```
public Student ( Student student) {
    this.name = student.name;
    this.age = student.age;
    this.exam = new Exam (student.exam);
}
```

→ The above structure helps us create a clean deep copy of the student object.

Static

→ They are properties of a class, and not the object
(For example, a static variable to track number of objects created for a class)

→ Static methods can not reference static fields and instances directly

```
class Test {
    int x;

    static void print() {
        System.out.println(x); // Not allowed
    }

    static void printIndirectly(Test test) {
        System.out.println(test.x); // works fine
    }
}
```

→ static block is a block of code that is executed only once, when the class is loaded, before objects are created and main() is run

→ Use → init. static variables
→ doing one time setup
→ complex static init, which is complex in one line (eg. database connection)

```
class Config {
    static String url;
    static int port;

    static {
        url = "localhost";
        port = 8000;
    }
}
```

→ Static blocks are executed in the order in which they are written in the class

→ Instance blocks are run each time an object is created, while static block runs only once at the time of class loading